



COMPUTER NETWORKS

TEAM NETWORKS

Department of Computer Science and Engineering

COMPUTER NETWORKS

Application Layer

Department of Computer Science and Engineering

Unit – 2 Application Layer

2.3 The Domain Name System

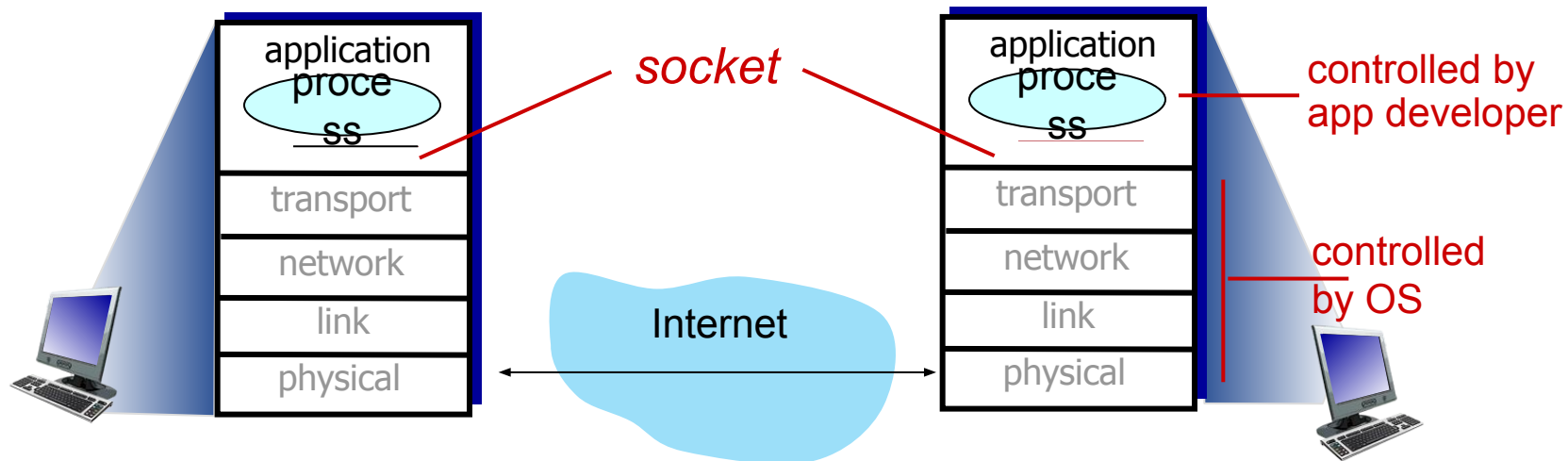
2.4 P2P Applications

2.5 Socket Programming with TCP & UDP

2.6 Other Application Layer Protocols

goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Two socket types for two transport services:

- *UDP*: unreliable datagram
- *TCP*: reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

UDP: no “connection” between client & server

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes (“datagrams”) between client and server



server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client



create
clientSocket =
socket.
socket(AF_INET,SOCK_DGRAM)

Create datagram with server IP and
port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket



Python UDPClient

include Python's socket library → `from socket import *`

`serverName = 'hostname'`

`serverPort = 12000`

create UDP socket for server →

`clientSocket = socket(AF_INET,
SOCK_DGRAM)`

get user keyboard input →

attach server name, port to message; send into socket →

`message = raw_input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),`

read reply characters from socket into string →

`(serverName, serverPort))`

print out received string and close socket →

`modifiedMessage, serverAddress =
clientSocket.recvfrom(2048)
print modifiedMessage.decode()
clientSocket.close()`

Python UDPServer

```
from socket import *
```

```
serverPort = 12000
```

create UDP socket →

```
serverSocket = socket(AF_INET, SOCK_DGRAM)
```

bind socket to local port number 12000 →

```
serverSocket.bind("", serverPort))
```

loop forever →

```
print ("The server is ready to receive")
```

Read from UDP socket into message, getting client's address (client IP and port) →

while True:

send upper case string back to this client →

```
message, clientAddress = serverSocket.recvfrom(2048)
```

```
modifiedMessage = message.decode().upper()
```

```
serverSocket.sendto(modifiedMessage.encode(),
                    clientAddress)
```

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

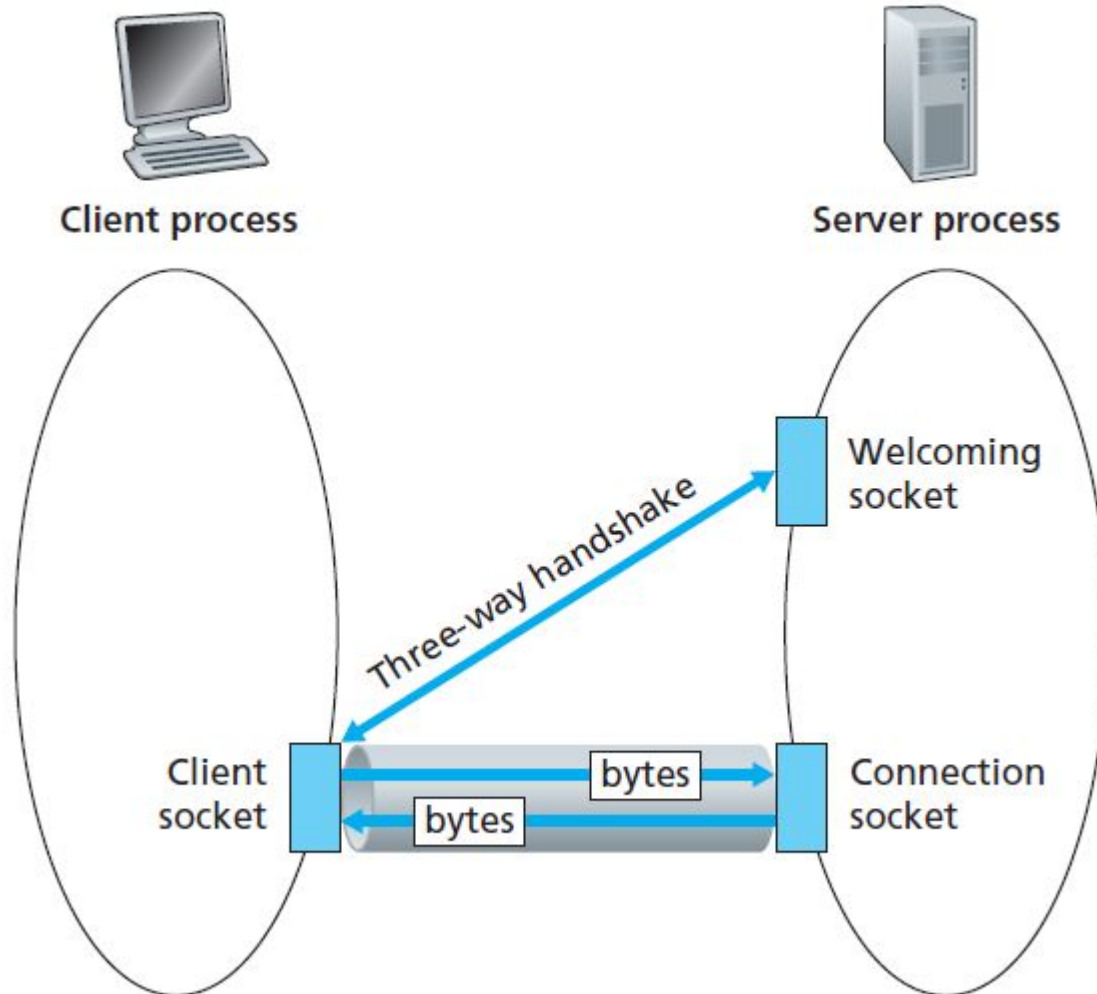
Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - source port numbers used to distinguish clients (more in Chap 3)

Application viewpoint

TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server





server (running on `hostid`)

client



create socket,
port=`x`, for incoming
request:
`serverSocket = socket()`

wait for incoming
connection request
`connectionSocket =`
`serverSocket.accept()`

read request
from
`connectionSocket`
et
write reply to
`connectionSocket`
et
close
`connectionSocket`
et

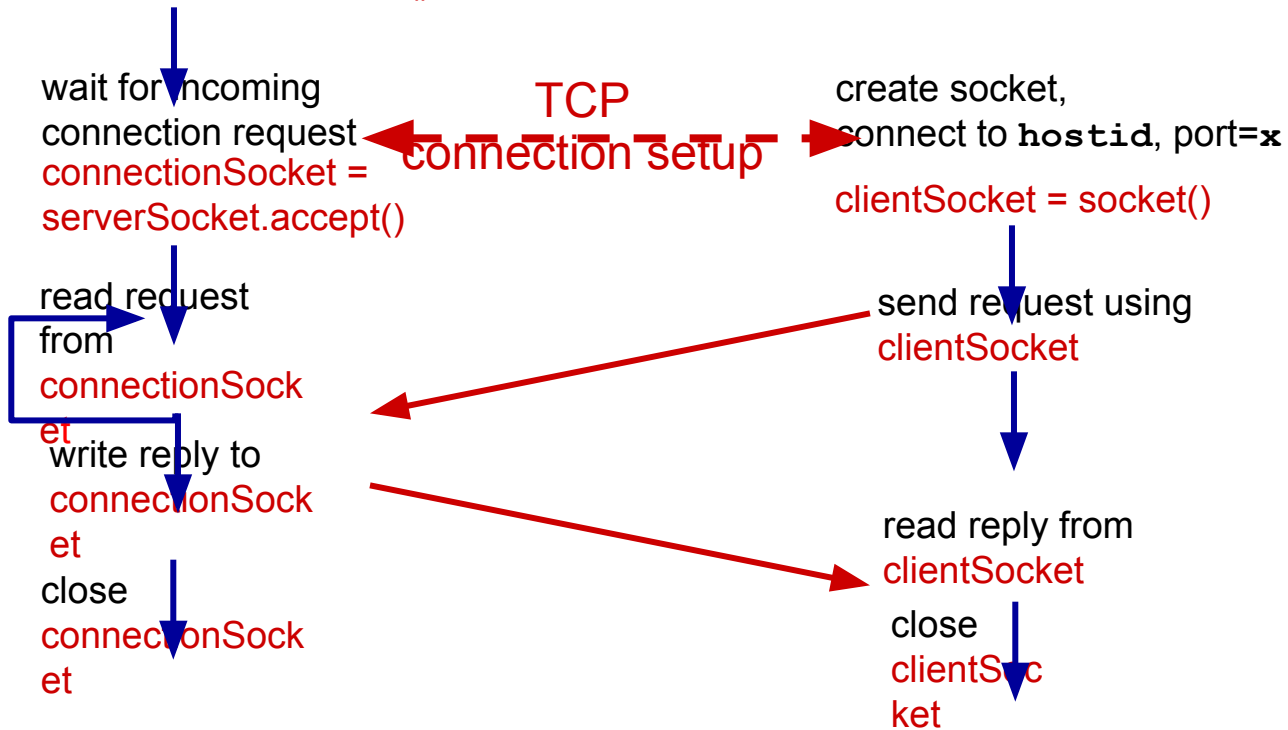
TCP

connection setup

create socket,
connect to `hostid`, port=`x`
`clientSocket = socket()`

send request using
`clientSocket`

read reply from
`clientSocket`
close
`clientSocket`



Python TCPClient

```
from socket import *
```

```
serverName = 'servername'
```

```
serverPort = 12000
```

create TCP socket for
server, remote port
12000

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

```
clientSocket.connect((serverName,serverPort))
```

```
sentence = raw_input('Input lowercase sentence:')
```

No need to attach
server name, port

```
clientSocket.send(sentence.encode())
```

```
modifiedSentence = clientSocket.recv(1024)
```

```
print ('From Server:', modifiedSentence.decode())
```

```
clientSocket.close()
```

Python TCPServer

	<pre>from socket import *</pre>
	<pre>serverPort = 12000</pre>
create TCP welcoming socket →	<pre>serverSocket = socket(AF_INET,SOCK_STREAM)</pre>
	<pre>serverSocket.bind(('',serverPort))</pre>
server begins listening for incoming TCP requests →	<pre>serverSocket.listen(1)</pre>
	<pre>print 'The server is ready to receive'</pre>
loop forever →	<pre>while True:</pre>
server waits on accept() for incoming requests, new socket created on return →	<pre>connectionSocket, addr = serverSocket.accept()</pre>
read bytes from socket (but not address as in UDP) →	<pre>sentence = connectionSocket.recv(1024).decode()</pre>
	<pre>capitalizedSentence = sentence.upper()</pre>
	<pre>connectionSocket.send(capitalizedSentence.encode())</pre>
close connection to this client (but <i>not</i> welcoming socket) →	<pre>connectionSocket.close()</pre>



THANK YOU

TEAM NETWORKS

Department of Computer Science and Engineering